

Objetos, arreglos y colecciones en JavaScript

Objetos, arreglos lineales y bidimensionales, métodos funcionales, clases, funciones flecha, JSON, Map, Set e inmutabilidad práctica

SEMANA	SESIONES	DURACIÓN	MODALIDAD
04	07 y 08	4 horas	Presencial

CASO PRÁCTICO

Gestor inteligente de inventario

Construirás una aplicación que registra, busca, filtra, ordena y elimina productos; calcula indicadores; organiza categorías con Set; localiza elementos mediante Map; y exporta e importa el estado con JSON.

Ruta del laboratorio

La guía alterna conceptos, experimentos cortos y construcción incremental. Ejecuta los bloques en orden y verifica el resultado después de cada etapa.

Bloque	Actividad	Tiempo	Evidencia
A	Comprender objetos, referencias y copias	25 min	Ejercicios comentados
B	Trabajar arreglos lineales y bidimensionales	30 min	Operaciones verificadas
C	Aplicar iteraciones y métodos funcionales	35 min	Transformaciones correctas
D	Usar clases, flechas, JSON, Map y Set	35 min	Mini casos resueltos
E	Construir HTML semántico	25 min	Interfaz y formulario
F	Aplicar estilos responsivos	20 min	Diseño funcional
G	Programar el gestor de inventario	55 min	Aplicación completa
H	Probar, depurar y ampliar	15 min	Matriz y reto

Tiempo total estimado: 240 minutos (4 horas).

Ficha del laboratorio

Elemento	Descripción
Logro	Organiza y procesa colecciones de datos con objetos, arreglos, clases, JSON, Map, Set y funciones flecha.
Producto	Gestor inteligente de inventario desarrollado con HTML, CSS y JavaScript separados.
Estrategia	Experimentación, modelado de datos, programación funcional, construcción incremental y pruebas.
Requisitos	Visual Studio Code estable y navegador moderno con DevTools.
Conocimientos previos	Variables, funciones, condicionales, DOM, eventos, cadenas y expresiones regulares.

Resultados de aprendizaje observables

- Modela entidades mediante objetos y clases con propiedades, métodos y getters.
- Diferencia copia superficial de clonación profunda y reconoce el comportamiento por referencia.
- Selecciona métodos de arreglos según necesite recorrer, transformar, filtrar, buscar o acumular.
- Procesa arreglos bidimensionales mediante iteraciones anidadas y métodos funcionales.
- Emplea funciones flecha reconociendo sus diferencias respecto de las funciones tradicionales.
- Serializa y reconstruye datos con JSON sin asumir que conserva prototipos, Map o Set.
- Utiliza Map para búsquedas por clave y Set para garantizar unicidad.

Enfoque: No memorices métodos aislados. Identifica primero la transformación requerida y elige la operación que exprese mejor esa intención.

1. Objetos: modelar información relacionada

Un objeto agrupa pares propiedad–valor. Las propiedades pueden contener valores primitivos, otros objetos, arreglos o funciones; cuando una propiedad contiene una función se denomina método.

Código 1. Literal de objeto

```
const producto = {
  id: "PRD-1001",
  nombre: "Monitor 24 pulgadas",
  categoria: "Pantallas",
  precio: 899.9,
  stock: 4,
  calcularValor() {
    return this.precio * this.stock;
  }
};

console.log(producto.nombre);
console.log(producto["categoria"]);
console.log(producto.calcularValor());
```

Notación	Cuándo usarla	Ejemplo
Punto	Nombre de propiedad conocido y válido como identificador	producto.precio
Corchetes	Propiedad dinámica, con espacios o calculada	producto[clave]
Método	Comportamiento relacionado con el objeto	producto.calcularValor()

1.1 Propiedades abreviadas, calculadas y desestructuración

Código 2. Sintaxis moderna de objetos

```
const nombre = "Teclado";
const precio = 219.5;
const claveStock = "stock";

const item = { nombre, precio, [claveStock]: 12 };
const { nombre: nombreProducto, stock = 0 } = item;

console.log(nombreProducto, stock);
```

La desestructuración extrae propiedades en variables. El alias evita conflictos de nombres y el valor predeterminado solo se aplica cuando la propiedad es undefined, no cuando vale 0 o null.

1.2 Referencias y copias superficiales

Los objetos se asignan por referencia. Dos variables pueden apuntar al mismo objeto. Spread y Object.assign crean una copia superficial: copian el primer nivel, pero los objetos anidados continúan compartidos.

Código 3. Referencia frente a copia

```
const original = {
  nombre: "Laptop",
  proveedor: { ciudad: "Lima" }
};

const alias = original;
const copia = { ...original };

alias.nombre = "Laptop Pro"; // cambia original
copia.proveedor.ciudad = "Cusco"; // también cambia el objeto anidado

console.log(original);
```

Clonación: `structuredClone(valor)` crea una copia profunda de muchos tipos incorporados, pero no clona funciones ni nodos DOM y no reemplaza un diseño de datos consciente.

1.3 Inspección segura de propiedades

Operación	Propósito
<code>Object.keys(objeto)</code>	Devuelve nombres de propiedades enumerables propias.
<code>Object.values(objeto)</code>	Devuelve sus valores enumerables propios.
<code>Object.entries(objeto)</code>	Devuelve pares [clave, valor], útiles para iterar.
<code>Object.hasOwn(objeto, clave)</code>	Comprueba si la propiedad pertenece directamente al objeto.
<code>objeto?.propiedad</code>	Detiene el acceso si el receptor es null o undefined.
<code>valor ?? alternativa</code>	Usa la alternativa solo ante null o undefined.

Código 4. Encadenamiento opcional

```
const configuracion = { tema: { color: "azul" } };

const color = configuracion.tema?.color ?? "predeterminado";
const idioma = configuracion.usuario?.idioma ?? "es-PE";

console.log({ color, idioma });
```

2. Arreglos lineales

Un arreglo es un objeto indexado desde cero, redimensionable y capaz de contener valores de distintos tipos. En una aplicación profesional conviene mantener elementos homogéneos para simplificar reglas y pruebas.

Código 5. Creación, acceso y comprobación

```
const productos = ["Monitor", "Teclado", "Mouse"];

console.log(productos[0]); // Monitor
console.log(productos.at(-1)); // Mouse
console.log(productos.length); // 3
console.log(Array.isArray(productos)); // true
```

2.1 Métodos mutables y no mutables

Intención	Muta el original	Alternativa que crea copia
Agregar al final	push(valor)	[...arreglo, valor] o concat(valor)
Quitar al final	pop()	slice(0, -1)
Agregar/quitar posiciones	splice()	toSpliced()
Ordenar	sort()	toSorted()
Invertir	reverse()	toReversed()
Reemplazar una posición	arreglo[indice] = valor	with(indice, valor)

Código 6. Ordenamiento sin mutación

```
const precios = [120, 35, 80];
const ordenados = precios.toSorted((a, b) => a - b);

console.log(precios); // [120, 35, 80]
console.log(ordenados); // [35, 80, 120]
```

Copia superficial: Los métodos de copia generan un nuevo arreglo, pero si los elementos son objetos, ambos arreglos mantienen referencias a esos mismos objetos.

2.2 Arreglos bidimensionales

JavaScript no posee una matriz especial incorporada: se representa como un arreglo cuyos elementos son otros arreglos. Cada fila puede incluso tener distinta longitud, por lo que las dimensiones deben validarse si el problema exige una matriz rectangular.

Código 7. Matriz de existencias por sede

```
const existencias = [
  [4, 12, 0], // Lima
  [2, 5, 3],  // Arequipa
  [6, 1, 4]   // Trujillo
];

console.log(existencias[1][2]); // 3

const total = existencias
  .flat()
  .reduce((acumulado, stock) => acumulado + stock, 0);

console.log(total); // 37
```

1. Cambia el stock de Arequipa para el segundo producto a 7.
2. Calcula el total de cada sede mediante map() y reduce().
3. Comprueba con every() que ninguna fila esté vacía.

3. Iteraciones y métodos funcionales

Los métodos iterativos reciben una función callback. El callback suele recibir elemento, índice y arreglo. No todos los métodos tienen el mismo objetivo ni deben usarse solo porque admiten una función flecha.

Método	Pregunta que responde	Retorno
forEach	¿Qué efecto ejecutaré por cada elemento?	undefined
map	¿Cómo transformo cada elemento?	Nuevo arreglo de igual longitud
filter	¿Qué elementos cumplen una condición?	Nuevo arreglo
find / findLast	¿Cuál es el primer/último elemento que cumple?	Elemento o undefined
some	¿Al menos uno cumple?	Boolean
every	¿Todos cumplen?	Boolean
reduce	¿Cómo acumulo un resultado único?	Cualquier valor acumulado
flat / flatMap	¿Cómo reduzco niveles o transformo y aplano?	Nuevo arreglo

Código 8. Cadena de transformación

```
const catalogo = [
  { nombre: "Monitor", precio: 899.9, stock: 4 },
  { nombre: "Mouse", precio: 129.9, stock: 0 },
  { nombre: "Teclado", precio: 219.5, stock: 12 }
];

const nombresDisponibles = catalogo
  .filter(producto => producto.stock > 0)
  .sorted((a, b) => a.precio - b.precio)
  .map(producto => producto.nombre);

console.log(nombresDisponibles);
```

3.1 reduce() con acumulador explícito

Código 9. Resumen de inventario

```
const resumen = catalogo.reduce((acumulado, producto) => ({
  unidades: acumulado.unidades + producto.stock,
  valor: acumulado.valor + producto.precio * producto.stock
}), { unidades: 0, valor: 0 });

console.log(resumen);
```

Buena práctica: Proporciona un valor inicial a reduce(). Hace explícito el tipo del acumulador y evita errores con arreglos vacíos.

3.2 Elegir entre bucles y métodos

Situación	Opción recomendada
Necesitas break, continue o await secuencial	for...of
Necesitas el índice y control total	for clásico

Situación	Opción recomendada
Solo ejecutarás un efecto por elemento	forEach()
Construirás un arreglo transformado	map()
Construirás un subconjunto	filter()
Buscarás un elemento y quieres detenerte	find()
Acumularás un número, objeto, Map u otra estructura	reduce()

4. Funciones flecha

La expresión de función flecha ofrece sintaxis compacta y captura this del ámbito exterior. No es un operador ni un reemplazo universal de function.

Código 10. Variantes de sintaxis

```
const duplicar = numero => numero * 2;
const sumar = (a, b) => a + b;
const crearProducto = (nombre, precio) => ({ nombre, precio });
const describir = producto => {
  const estado = producto.stock > 0 ? "disponible" : "agotado";
  return `${producto.nombre}: ${estado}`;
};
```

Característica	Flecha	function tradicional
this propio	No; usa el del exterior	Sí, depende de cómo se invoca
arguments propio	No	Sí
Uso con new	No es constructora	Puede ser constructora
Retorno implícito	Sí, con una expresión	No
Método de objeto con this dinámico	Generalmente no conviene	Sí
Callback breve	Muy adecuada	También posible

Error típico: Para devolver un objeto literal de forma implícita, envuélvelo entre paréntesis: elemento => ({ id: elemento.id }).

5. Clases y objetos

Las clases ofrecen una sintaxis clara sobre el sistema de prototipos de JavaScript. El constructor inicializa cada instancia; los métodos se comparten mediante el prototipo y no se copian dentro de cada objeto.

Código 11. Clase Producto

```

class Producto {
  constructor({ id, nombre, precio, stock }) {
    this.id = String(id);
    this.nombre = String(nombre);
    this.precio = Number(precio);
    this.stock = Number(stock);
  }

  get valorInventario() {
    return this.precio * this.stock;
  }

  static desdeObjeto(datos) {
    return new Producto(datos);
  }
}

const monitor = new Producto({
  id: "PRD-1001", nombre: "Monitor", precio: 899.9, stock: 4
});

```

Elemento	Responsabilidad
constructor	Inicializa el estado propio de una instancia nueva.
Método de instancia	Opera sobre una instancia y se invoca con objeto.metodo().
getter	Expone un valor calculado con sintaxis de propiedad.
Método static	Pertenece a la clase y suele crear, validar o coordinar instancias.
Campo privado #campo	Solo puede accederse dentro del cuerpo de la clase.

Modelo del proyecto: La clase Producto reconstruye instancias después de JSON.parse(), porque JSON conserva datos, pero no el prototipo ni sus getters y métodos.

6. JSON: representar e intercambiar datos

JSON es un formato textual, no una clase ni una base de datos. Sus valores admitidos son objeto, arreglo, cadena, número finito, boolean y null. Los nombres de propiedades y cadenas usan comillas dobles; no admite comentarios ni comas finales.

JavaScript	JSON
{ nombre: 'Ana' }	{"nombre":"Ana"}
undefined	No existe como valor JSON
NaN / Infinity	Se serializan como null dentro de estructuras
Date	Normalmente se convierte en cadena ISO
Map / Set	No se representan automáticamente como sus entradas
BigInt	JSON.stringify lanza TypeError sin conversión personalizada

JavaScript	JSON
Métodos y prototipo	No se conservan

Código 12. Serializar y analizar

```
const datos = [
  { id: "PRD-1001", nombre: "Monitor", precio: 899.9 }
];

const textoJson = JSON.stringify(datos, null, 2);
const recuperados = JSON.parse(textoJson);

console.log(typeof textoJson); // string
console.log(Array.isArray(recuperados)); // true
```

Seguridad: Nunca uses `eval()` para leer JSON. `JSON.parse()` analiza la gramática del formato; después debes validar estructura, tipos, rangos y reglas de negocio.

6.1 replacer y revive

Código 13. Conversión controlada de fechas

```
const pedido = { creado: new Date("2026-09-04T10:00:00Z") };
const texto = JSON.stringify(pedido);

const copia = JSON.parse(texto, (clave, valor) =>
  clave === "creado" ? new Date(valor) : valor
);

console.log(copia.creado instanceof Date); // true
```

El segundo argumento de `stringify` es `replacer`; el segundo de `parse` es `reviver`. Úsalos con reglas claras y pruebas, ya que una conversión automática basada solo en apariencia puede transformar datos que debían permanecer como texto.

7. Map y Set

Map almacena pares clave–valor y acepta claves de cualquier tipo. Set almacena valores únicos. Ambos conservan el orden de inserción al iterar y exponen `size` en lugar de `length`.

Necesidad	Estructura	Operaciones principales
Buscar producto por id	Map	set, get, has, delete, clear, size
Mantener categorías únicas	Set	add, has, delete, clear, size
Lista ordenada y transformable	Array	map, filter, find, reduce, toSorted
Entidad con campos conocidos	Object o class	Propiedades, métodos, Object.keys

Código 14. Índice Map y categorías Set

```
const productos = [
  { id: "P1", categoria: "Pantallas" },
  { id: "P2", categoria: "Periféricos" },
  { id: "P3", categoria: "Periféricos" }
];

const porId = new Map(productos.map(producto => [producto.id, producto]));
const categorias = new Set(productos.map(producto => producto.categoria));

console.log(porId.get("P2"));
console.log([...categorias]);
```

7.1 Igualdad, identidad y serialización

Set y Map comparan claves con SameValueZero: NaN coincide con NaN y 0 con -0. Los objetos se comparan por identidad, por lo que dos literales visualmente iguales son claves distintas.

Código 15. Identidad de objetos

```
const conjunto = new Set();
conjunto.add({ id: 1 });
conjunto.add({ id: 1 });

console.log(conjunto.size); // 2: son referencias diferentes

const mapaComoJson = JSON.stringify([...porId]);
const setComoJson = JSON.stringify([...categorias]);
```

8. Proyecto integrador

El proyecto separa estructura, presentación y comportamiento. No requiere bibliotecas externas ni servidor: puede abrirse como archivo local en un navegador moderno.

Archivo	Responsabilidad
index.html	Formulario, filtros, tabla, indicadores, controles JSON y atributos de accesibilidad.
css/styles.css	Sistema visual, cuadrículas responsivas, tabla, estados de foco y mensajes.
js/app.js	Modelo Producto, estado, transformaciones, Map/Set, JSON, DOM y eventos.

1. Crea una carpeta llamada lab-semana-04.
2. Dentro crea index.html, la carpeta css con styles.css y la carpeta js con app.js.
3. Abre la carpeta completa en Visual Studio Code.
4. Mantén DevTools abierto y revisa Console después de cada bloque JavaScript.

Estructura: lab-semana-04/index.html; lab-semana-04/css/styles.css; lab-semana-04/js/app.js.

9. Implementación de HTML

Abre index.html y escribe los bloques en el orden presentado. Guarda y comprueba el navegador después de cada bloque.

Código 16. index.html — bloque 1 de 7

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gestor inteligente de inventario</title>
  <link rel="stylesheet" href="./css/styles.css">
  <script src="./js/app.js" defer></script>
</head>
<body>
  <header class="encabezado">
    <div class="contenedor">
      <p class="etiqueta">JavaScript Avanzado · Semana 4</p>
      <h1>Gestor inteligente de inventario</h1>
      <p class="introduccion">
        Objetos, arreglos, clases, colecciones y JSON aplicados a una solución web.
      </p>
    </div>
  </header>

  <main class="contenedor contenido">

```

doctype, idioma, viewport y defer preparan una página moderna y aseguran que el DOM exista antes de ejecutar JavaScript.

Código 17. index.html — bloque 2 de 7

```

<section class="panel" aria-labelledby="titulo-registro">
  <h2 id="titulo-registro">Registrar producto</h2>
  <form id="formProducto" novalidate>
    <div class="campos">
      <div class="campo campo-amplio">
        <label for="nombre">Nombre</label>
        <input id="nombre" name="nombre" type="text" maxlength="60"
          placeholder="Teclado mecánico" required>
      </div>

      <div class="campo">
        <label for="categoria">Categoría</label>
        <input id="categoria" name="categoria" type="text" maxlength="30"
          placeholder="Periféricos" required>
      </div>

      <div class="campo">
        <label for="precio">Precio (S/)</label>
        <input id="precio" name="precio" type="number" min="0.01"
          max="999999.99" step="0.01" placeholder="149.90" required>
      </div>

```

El formulario asocia cada label con su control, usa tipos numéricos y mantiene la validación de negocio en JavaScript.

Código 18. index.html — bloque 3 de 7

```

    <div class="campo">
      <label for="stock">Stock</label>
      <input id="stock" name="stock" type="number" min="0"
        max="99999" step="1" placeholder="12" required>
    </div>
  </div>

  <div class="acciones">
    <button type="submit">Agregar producto</button>
    <button id="btnRestaurar" type="button" class="secundario">Restaurar datos</button>
  </div>
</form>
<div id="mensajes" class="mensajes" role="alert" hidden></div>
</section>

<section class="panel" aria-labelledby="titulo-catalogo">
  <div class="cabecera-seccion">
    <div>
      <p class="etiqueta">Catálogo</p>
      <h2 id="titulo-catalogo">Productos registrados</h2>

```

Los filtros separan búsqueda, categoría y orden; cada control representa una decisión independiente.

Código 19. index.html — bloque 4 de 7

```

    </div>
    <span id="resumenVisible" class="insignia">0 productos</span>
  </div>

  <div class="filtros">
    <div class="campo campo-amplio">
      <label for="busqueda">Buscar</label>
      <input id="busqueda" type="search" placeholder="Nombre o categoría">
    </div>
    <div class="campo">
      <label for="filtroCategoria">Categoría</label>
      <select id="filtroCategoria">
        <option value="">Todas</option>
      </select>
    </div>
    <div class="campo">
      <label for="orden">Ordenar</label>
      <select id="orden">
        <option value="nombre">Nombre A-Z</option>
        <option value="precio-desc">Mayor precio</option>
        <option value="stock-asc">Menor stock</option>

```

La tabla usa thead y tbody, permite desplazamiento horizontal y deja sus filas al renderizador JavaScript.

Código 20. index.html — bloque 5 de 7

```

        </select>
      </div>
    </div>

    <div class="tabla-contenedor">
      <table>
        <thead>
          <tr>
            <th>Producto</th>
            <th>Categoría</th>
            <th class="numero">Precio</th>
            <th class="numero">Stock</th>
            <th class="numero">Valor</th>
            <th>Acción</th>
          </tr>
        </thead>
        <tbody id="cuerpoInventario"></tbody>
      </table>
    </div>
    <p id="estadoVacio" class="estado-vacio" hidden>No se encontraron productos.</p>
  </section>

```

Los indicadores usan dl para pares término–valor y role=status para comunicar el estado del stock.

Código 21. index.html — bloque 6 de 7

```

<section class="panel" aria-labelledby="titulo-resumen">
  <h2 id="titulo-resumen">Resumen del inventario</h2>
  <dl class="metricas">
    <div><dt>Productos</dt><dd id="totalProductos">0</dd></div>
    <div><dt>Unidades</dt><dd id="totalUnidades">0</dd></div>
    <div><dt>Valor total</dt><dd id="valorTotal">$ 0.00</dd></div>
    <div><dt>Stock bajo</dt><dd id="stockBajo">0</dd></div>
  </dl>
  <p id="alertaStock" class="alerta" role="status"></p>
</section>

<section class="panel" aria-labelledby="titulo-json">
  <h2 id="titulo-json">Intercambio mediante JSON</h2>
  <p>Exporta los objetos actuales o pega un arreglo JSON válido para reconstruir el inventario.</p>
  <label for="areaJson">Datos JSON</label>
  <textarea id="areaJson" rows="10" spellcheck="false"
    placeholder='[{"id":"PRD-1001","nombre":"Monitor","categoria":"Pantallas","precio":899.9,"stock":4}]'></textarea>
  <div class="acciones">
    <button id="btnExportar" type="button">Exportar</button>
    <button id="btnImportar" type="button" class="secundario">Importar</button>
  </div>

```

El área JSON no ejecuta contenido: solo recibe texto que después será analizado y validado.

Código 22. index.html — bloque 7 de 7

```

    </div>
  </section>
</main>
</body>
</html>

```

10. Implementación de CSS

Abre `css/styles.css` y escribe los bloques en el orden presentado. Guarda y comprueba el navegador después de cada bloque.

Código 23. `css/styles.css` — bloque 1 de 6

```
:root {
  color-scheme: light;
  --azul-900: #0b2545;
  --azul-700: #1f4d78;
  --azul-500: #2e74b5;
  --azul-100: #e8f1fb;
  --rojo: #e63b2e;
  --tinta: #172033;
  --gris: #667085;
  --borde: #d7dee8;
  --fondo: #f4f7fb;
  --blanco: #ffffff;
  --verde: #13795b;
  --error: #9b1c1c;
  --advertencia: #8a6500;
}

* { box-sizing: border-box; }

body {
  margin: 0;
```

Las variables concentran colores y facilitan mantener una identidad visual consistente.

Código 24. `css/styles.css` — bloque 2 de 6

```
font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
color: var(--tinta);
background: var(--fondo);
line-height: 1.5;
}

.contenedor {
  width: min(1180px, calc(100% - 2rem));
  margin-inline: auto;
}

.encabezado {
  padding: 2.7rem 0 5rem;
  color: var(--blanco);
  background: linear-gradient(135deg, var(--azul-900), var(--azul-500));
}

h1, h2, p { margin-top: 0; }
h1 { max-width: 820px; margin-bottom: .5rem; font-size: clamp(2rem, 5vw, 3.5rem); line-height: 1.05; }
h2 { margin-bottom: 1rem; color: var(--azul-900); }
.introduccion { max-width: 720px; margin-bottom: 0; color: #dbeafe; }
```

La cuadrícula principal distribuye paneles; el catálogo ocupa todo el ancho por contener datos tabulares.

Código 25. css/styles.css — bloque 3 de 6

```
.etiqueta { margin-bottom: .35rem; color: var(--rojo); font-size: .78rem; font-weight: 800; letter-spacing: .08em; text-transform: uppercase; }

.contenido {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 1.25rem;
  margin-top: -2.8rem;
  padding-bottom: 3rem;
}

.panel {
  padding: clamp(1.1rem, 3vw, 1.8rem);
  border: 1px solid var(--borde);
  border-radius: 1rem;
  background: var(--blanco);
  box-shadow: 0 18px 45px rgb(11 37 69 / 10%);
}

.panel:nth-child(2) { grid-column: 1 / -1; }
.campos, .filtros { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: .9rem; }
.campo-amplio { grid-column: span 2; }
```

Los controles comparten geometría y conservan un indicador de foco visible para teclado.

Código 26. css/styles.css — bloque 4 de 6

```
.campo { margin-bottom: .9rem; }
label { display: block; margin-bottom: .35rem; font-weight: 700; }
input, select, textarea {
  width: 100%;
  padding: .72rem .8rem;
  border: 1px solid #aeb9c8;
  border-radius: .55rem;
  color: inherit;
  background: #fff;
  font: inherit;
}
textarea { resize: vertical; font-family: ui-monospace, "Cascadia Code", monospace; font-size: .9rem; }
input:focus, select:focus, textarea:focus { outline: 3px solid rgb(46 116 181 / 22%); border-color: var(--azul-500); }

.acciones { display: flex; flex-wrap: wrap; gap: .7rem; margin-top: .5rem; }
button {
  padding: .7rem .95rem;
  border: 2px solid var(--azul-700);
  border-radius: .55rem;
  color: #fff;
  background: var(--azul-700);
}
```

La tabla usa números tabulares, desplazamiento horizontal y una clase específica para stock bajo.

Código 27. css/styles.css — bloque 5 de 6

```

font: inherit;
font-weight: 750;
cursor: pointer;
}
button:hover { background: var(--azul-900); }
button:focus-visible { outline: 3px solid rgb(230 59 46 / 35%); outline-offset: 2px; }
button.secundario { color: var(--azul-700); background: #fff; }
button.peligro { padding: .35rem .55rem; border-color: var(--error); color: var(--error); background: #fff; font-size: .84rem; }

.mensajes { margin-top: 1rem; padding: .8rem 1rem; border-left: 4px solid var(--error); color: var(--error); background: #fdecec; }
.mensajes.exito { border-color: var(--verde); color: var(--verde); background: #e8f5f0; }
.mensajes ul { margin: .35rem 0 0; padding-left: 1.2rem; }
.cabecera-seccion { display: flex; align-items: start; justify-content: space-between; gap: 1rem; }
.insignia { padding: .35rem .65rem; border-radius: 999px; color: var(--azul-900); background: var(--azul-100); font-weight: 750; white-space: nowrap; }

.tabla-contenedor { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: .7rem .6rem; border-bottom: 1px solid var(--borde); text-align: left; }
th { color: var(--azul-900); background: var(--azul-100); font-size: .87rem; }
.numero { text-align: right; font-variant-numeric: tabular-nums; }
.stock-bajo { color: var(--advertencia); font-weight: 800; }

```

Las media queries reorganizan paneles, campos e indicadores sin cambiar el contenido HTML.

Código 28. css/styles.css — bloque 6 de 6

```

.estado-vacio { padding: 1rem; color: var(--gris); text-align: center; }

.metricas { display: grid; grid-template-columns: repeat(4, 1fr); gap: .65rem; margin: 0; }
.metricas div { padding: .8rem; border-radius: .65rem; background: var(--azul-100); text-align: center; }
.metricas dt { color: var(--gris); font-size: .78rem; }
.metricas dd { margin: 0; color: var(--azul-900); font-size: 1.35rem; font-weight: 800; }
.alerta { margin: 1rem 0 0; color: var(--advertencia); font-weight: 700; }
[hidden] { display: none !important; }

@media (max-width: 820px) {
  .contenido, .campos, .filtros { grid-template-columns: 1fr; }
  .panel:nth-child(2), .campo-amplio { grid-column: auto; }
  .metricas { grid-template-columns: repeat(2, 1fr); }
}

@media (max-width: 480px) {
  .metricas { grid-template-columns: 1fr; }
  .cabecera-seccion { flex-direction: column; }
}

```

11. Implementación de JavaScript

Abre `js/app.js` y escribe los bloques en el orden presentado. Guarda y comprueba el navegador después de cada bloque.

Código 29. js/app.js — bloque 1 de 16

```
"use strict";

class Producto {
  constructor({ id, nombre, categoria, precio, stock }) {
    this.id = String(id);
    this.nombre = String(nombre).trim();
    this.categoria = String(categoria).trim();
    this.precio = Number(precio);
    this.stock = Number(stock);
  }

  get valorInventario() {
    return this.precio * this.stock;
  }

  get tieneStockBajo() {
    return this.stock > 0 && this.stock <= 5;
  }
}
```

Producto encapsula conversión de datos, valores calculados, serialización y reconstrucción de instancias.

Código 30. js/app.js — bloque 2 de 16

```
toJSON() {
  return {
    id: this.id,
    nombre: this.nombre,
    categoria: this.categoria,
    precio: this.precio,
    stock: this.stock
  };
}

static desdeObjeto(datos) {
  return new Producto(datos);
}

const DATOS_INICIALES = [
  { id: "PRD-1001", nombre: "Monitor 24 pulgadas", categoria: "Pantallas", precio: 899.9, stock: 4 },
  { id: "PRD-1002", nombre: "Teclado mecánico", categoria: "Periféricos", precio: 219.5, stock: 12 },
  { id: "PRD-1003", nombre: "Mouse ergonómico", categoria: "Periféricos", precio: 129.9, stock: 0 },
  { id: "PRD-1004", nombre: "Base para laptop", categoria: "Accesorios", precio: 89.9, stock: 8 }
];
```

Los datos iniciales son objetos planos que se convierten en instancias; Intl formatea moneda y compara texto para Perú.

Código 31. js/app.js — bloque 3 de 16

```
const formateadorMoneda = new Intl.NumberFormat("es-PE", {
  style: "currency",
  currency: "PEN"
});

const comparadorTexto = new Intl.Collator("es-PE", {
  sensitivity: "base"
});

function normalizarTexto(texto) {
  return String(texto).trim().replace(/\s+/gu, " ");
}

function validarDatos(datos) {
  const errores = [];

  if (normalizarTexto(datos.nombre).length < 3) {
    errores.push("El nombre debe tener al menos 3 caracteres.");
  }
}
```

Las funciones puras normalizan y validan sin tocar el DOM, lo que facilita probarlas por separado.

Código 32. js/app.js — bloque 4 de 16

```
if (normalizarTexto(datos.categoria).length < 2) {
  errores.push("La categoría debe tener al menos 2 caracteres.");
}
if (!Number.isFinite(datos.precio) || datos.precio <= 0) {
  errores.push("El precio debe ser un número mayor que cero.");
}
if (!Number.isInteger(datos.stock) || datos.stock < 0) {
  errores.push("El stock debe ser un entero mayor o igual que cero.");
}

return errores;
}

function obtenerCategorias(productos) {
  const categoriasUnicas = new Set(productos.map(producto => producto.categoria));
  return [...categoriasUnicas].sorted(comparadorTexto.compare);
}

function crearIndice(productos) {
  return new Map(productos.map(producto => [producto.id, producto]));
}
```

Set elimina categorías repetidas y Map crea un índice por id a partir del arreglo principal.

Código 33. js/app.js — bloque 5 de 16

```
function filtrarProductos(productos, { texto = "", categoria = "" } = {}) {
  const termino = normalizarTexto(texto).toLocaleLowerCase("es-PE");

  return productos.filter(producto => {
    const coincideTexto = termino === "" ||
      producto.nombre.toLocaleLowerCase("es-PE").includes(termino) ||
      producto.categoria.toLocaleLowerCase("es-PE").includes(termino);
    const coincideCategoria = categoria === "" || producto.categoria === categoria;
    return coincideTexto && coincideCategoria;
  });
}

function ordenarProductos(productos, criterio) {
  const comparadores = {
    "precio-desc": (a, b) => b.precio - a.precio,
    "stock-asc": (a, b) => a.stock - b.stock,
    nombre: (a, b) => comparadorTexto.compare(a.nombre, b.nombre)
  };

  return productos.toSorted(comparadores[criterio] ?? comparadores.nombre);
}
```

filter combina condiciones de búsqueda y categoría; toSorted ordena una copia.

Código 34. js/app.js — bloque 6 de 16

```
}

function calcularResumen(productos) {
  return productos.reduce((resumen, producto) => ({
    productos: resumen.productos + 1,
    unidades: resumen.unidades + producto.stock,
    valor: resumen.valor + producto.valorInventario,
    stockBajo: resumen.stockBajo + (producto.tieneStockBajo ? 1 : 0)
  }), { productos: 0, unidades: 0, valor: 0, stockBajo: 0 });
}

function validarColeccionImportada(valor) {
  if (!Array.isArray(valor)) {
    throw new TypeError("El JSON debe contener un arreglo de productos.");
  }

  const productos = valor.map(Producto.desdeObjeto);
  const errores = productos.flatMap((producto, indice) =>
    validarDatos(producto).map(error => `Elemento ${indice + 1}: ${error}`)
  );
  const ids = productos.map(producto => producto.id);
}
```

reduce produce un único resumen sin depender de variables globales.

Código 35. js/app.js — bloque 7 de 16

```

    if (new Set(ids).size !== ids.length) {
        errores.push("Los identificadores no pueden repetirse.");
    }
    if (productos.some(producto => producto.id.trim() === "")) {
        errores.push("Todos los productos deben incluir un identificador.");
    }
    if (errores.length > 0) {
        throw new TypeError(errores.join(" "));
    }

    return productos;
}

// --- Interfaz del navegador ---

const formulario = document.querySelector("#formProducto");
const campoNombre = document.querySelector("#nombre");
const campoCategoria = document.querySelector("#categoria");
const campoPrecio = document.querySelector("#precio");
const campoStock = document.querySelector("#stock");

```

La importación valida que exista un arreglo, reconstruye instancias y rechaza ids duplicados antes de modificar el estado.

Código 36. js/app.js — bloque 8 de 16

```

const busqueda = document.querySelector("#busqueda");
const filtroCategoria = document.querySelector("#filtroCategoria");
const selectorOrden = document.querySelector("#orden");
const cuerpoInventario = document.querySelector("#cuerpoInventario");
const estadoVacio = document.querySelector("#estadoVacio");
const mensajes = document.querySelector("#mensajes");
const areaJson = document.querySelector("#areaJson");

let inventario = DATOS_INICIALES.map(Producto.desdeObjeto);
let indicePorId = crearIndice(inventario);
let correlativo = 1005;

function mostrarMensaje(texto, tipo = "error") {
    mensajes.textContent = texto;
    mensajes.classList.toggle("exito", tipo === "exito");
    mensajes.hidden = false;
}

function ocultarMensaje() {
    mensajes.hidden = true;
    mensajes.textContent = "";
}

```

Las referencias DOM se almacenan una vez; el estado principal permanece en inventario y las estructuras derivadas se reconstruyen.

Código 37. js/app.js — bloque 9 de 16

```

}

function crearCelda(texto, clase = "") {
  const celda = document.createElement("td");
  celda.textContent = texto;
  if (clase) celda.className = clase;
  return celda;
}

function crearFila(producto) {
  const fila = document.createElement("tr");
  const claseStock = producto.tieneStockBajo ? "numero stock-bajo" : "numero";
  const boton = document.createElement("button");

  fila.append(
    crearCelda(producto.nombre),
    crearCelda(producto.categoria),
    crearCelda(formateadorMoneda.format(producto.precio), "numero"),
    crearCelda(String(producto.stock), claseStock),
    crearCelda(formateadorMoneda.format(producto.valorInventario), "numero")
  );
};

```

Las filas se crean con `textContent` y nodos explícitos; `data-id` conecta el botón con el Map.

Código 38. js/app.js — bloque 10 de 16

```

  boton.type = "button";
  boton.className = "peligro";
  boton.dataset.id = producto.id;
  boton.textContent = "Eliminar";
  const celdaAccion = document.createElement("td");
  celdaAccion.append(boton);
  fila.append(celdaAccion);
  return fila;
}

function actualizarCategorias() {
  const seleccionActual = filtroCategoria.value;
  filtroCategoria.replaceChildren(new Option("Todas", ""));

  obtenerCategorias(inventario).forEach(categoria => {
    filtroCategoria.add(new Option(categoria, categoria));
  });
};

```

Las categorías se regeneran desde Set y conservan la selección únicamente si todavía existe.

Código 39. js/app.js — bloque 11 de 16

```

    filtroCategoria.value = obtenerCategorias(inventario).includes(seleccionActual)
      ? seleccionActual
      : "";
  }

  function actualizarResumen() {
    const resumen = calcularResumen(inventario);
    document.querySelector("#totalProductos").textContent = resumen.productos;
    document.querySelector("#totalUnidades").textContent = resumen.unidades;
    document.querySelector("#valorTotal").textContent = formateadorMoneda.format(resumen.valor);
    document.querySelector("#stockBajo").textContent = resumen.stockBajo;

    const hayAgotados = inventario.some(producto => producto.stock === 0);
    const alerta = document.querySelector("#alertaStock");
    alerta.textContent = hayAgotados
      ? "Atención: existe al menos un producto agotado."
      : "No existen productos agotados.";
  }

```

renderizar deriva productos visibles sin alterar el orden del inventario original.

Código 40. js/app.js — bloque 12 de 16

```

function renderizar() {
  const filtrados = filtrarProductos(inventario, {
    texto: busqueda.value,
    categoria: filtroCategoria.value
  });
  const visibles = ordenarProductos(filtrados, selectorOrden.value);

  cuerpoInventario.replaceChildren(...visibles.map(crearFila));
  estadoVacio.hidden = visibles.length > 0;
  document.querySelector("#resumenVisible").textContent =
    `${visibles.length} producto${visibles.length === 1 ? "" : "s"}`;
  actualizarResumen();
}

function sincronizarEstructuras() {
  indicePorId = crearIndice(inventario);
  actualizarCategorias();
  renderizar();
}

```

El registro valida antes de crear la instancia y agrega mediante spread para producir un arreglo nuevo.

Código 41. js/app.js — bloque 13 de 16

```
function manejarRegistro(evento) {
  evento.preventDefault();
  ocultarMensaje();

  const datos = {
    id: `PRD-${correlativo}`,
    nombre: normalizarTexto(campoNombre.value),
    categoria: normalizarTexto(campoCategoria.value),
    precio: Number(campoPrecio.value),
    stock: Number(campoStock.value)
  };
  const errores = validarDatos(datos);

  if (errores.length > 0) {
    mostrarMensaje(errores.join(" "));
    return;
  }

  inventario = [...inventario, new Producto(datos)];
  correlativo += 1;
  formulario.reset();
}
```

La eliminación usa delegación de eventos, `closest()`, `dataset` y una búsqueda Map de tiempo esperado constante.

Código 42. js/app.js — bloque 14 de 16

```
sincronizarEstructuras();
mostrarMensaje("Producto agregado correctamente.", "exito");
}

function manejarEliminacion(evento) {
  const boton = evento.target.closest("button[data-id]");
  if (!boton) return;

  const producto = indicePorId.get(boton.dataset.id);
  if (!producto) return;

  inventario = inventario.filter(elemento => elemento.id !== producto.id);
  sincronizarEstructuras();
  mostrarMensaje(`${producto.nombre} fue eliminado.`, "exito");
}

function exportarJson() {
  areaJson.value = JSON.stringify(inventario, null, 2);
  mostrarMensaje("Inventario exportado como JSON.", "exito");
}
```

`JSON.stringify` genera texto legible; `JSON.parse` se mantiene dentro de `try...catch` y la asignación ocurre tras validar.

Código 43. js/app.js — bloque 15 de 16

```
function importarJson() {
  try {
    const datos = JSON.parse(areaJson.value);
    inventario = validarColeccionImportada(datos);
    sincronizarEstructuras();
    mostrarMensaje("Inventario importado correctamente.", "exito");
  } catch (error) {
    mostrarMensaje(`No se pudo importar: ${error.message}`);
  }
}

function restaurarDatos() {
  inventario = DATOS_INICIALES.map(Producto.desdeObjeto);
  correlativo = 1005;
  areaJson.value = "";
  ocultarMensaje();
  sincronizarEstructuras();
}

formulario.addEventListener("submit", manejarRegistro);
cuerpoInventario.addEventListener("click", manejarEliminacion);
```

Los listeners conectan eventos con funciones nombradas y la sincronización inicial produce la primera vista.

Código 44. js/app.js — bloque 16 de 16

```
busqueda.addEventListener("input", renderizar);
filtroCategoria.addEventListener("change", renderizar);
selectorOrden.addEventListener("change", renderizar);
document.querySelector("#btnExportar").addEventListener("click", exportarJson);
document.querySelector("#btnImportar").addEventListener("click", importarJson);
document.querySelector("#btnRestaurar").addEventListener("click", restaurarDatos);

sincronizarEstructuras();
```

12. Ejecución y verificación

1. Guarda index.html, css/styles.css y js/app.js.
2. Abre index.html y confirma que aparecen cuatro productos iniciales.
3. Abre DevTools y verifica que Console no muestre errores.
4. Ejecuta la matriz completa; registra entrada, salida real y resultado de la comparación.
5. Usa breakpoints en manejarRegistro(), renderizar() e importarJson() cuando una prueba falle.

12.1 Matriz mínima de pruebas

Caso	Acción / entrada	Resultado esperado
1. Estado inicial	Abrir la aplicación	4 productos, 24 unidades y 1 producto con stock bajo
2. Registro válido	Webcam; Video; 250.00; stock 6	Nuevo producto, resumen actualizado e id PRD-1005
3. Precio inválido	Precio 0	Mensaje de validación; no se agrega
4. Stock decimal	Stock 2.5	Mensaje: stock entero

Caso	Acción / entrada	Resultado esperado
5. Búsqueda	Buscar periféricos	Se muestran Teclado y Mouse
6. Categoría	Filtrar Pantallas	Solo aparece Monitor
7. Orden	Mayor precio	Monitor aparece primero en los datos iniciales
8. Eliminar	Eliminar Mouse	Desaparece; Map y resumen se reconstruyen
9. Exportar	Pulsar Exportar	Textarea contiene un arreglo JSON legible
10. Importar	Exportar, eliminar y volver a importar	Se reconstruyen instancias y getters
11. JSON inválido	Pegar {sin comillas}	Error controlado; inventario anterior se conserva
12. Id duplicado	Duplicar un objeto en JSON	Importación rechazada

12.2 Evidencias de los conceptos

Concepto	Dónde se observa
Class y getter	Producto y valorInventario
Array de objetos	inventario
map	Creación de instancias, filas, categorías e índice
filter	Búsqueda, categoría y eliminación inmutable
reduce	calcularResumen
some	Detección de producto agotado
toSorted	Orden sin mutar el inventario
Set	Categorías únicas e ids duplicados
Map	Índice indicePorId y búsqueda por id
JSON	Exportación, análisis, validación y reconstrucción
Funciones flecha	Callbacks de métodos iterativos y eventos breves

12.3 Errores frecuentes

Síntoma	Causa probable	Corrección
sort cambia el catálogo	Se ordenó el arreglo original	Usa toSorted o copia antes de sort.
Métodos ausentes tras importar	JSON.parse devuelve objetos planos	Reconstruye con Producto.desdeObjeto.
Map exporta {}	JSON no serializa entradas de Map	Convierte con [...mapa] o Array.from().
Categorías repetidas	Se usó Array sin control de unicidad	Construye un Set y conviértelo a Array.
NaN en el resumen	Entrada no convertida o validada	Number(), Number.isFinite y reglas previas.

Síntoma	Causa probable	Corrección
this es undefined	Flecha usada como método con this dinámico	Usa sintaxis de método o function.
Botón no elimina	dataset o delegación incorrectos	Revisa closest, data-id e índice Map.
Importación destruye datos	Se asignó antes de validar	Analiza y valida en variable; asigna al final.

13. Reto de extensión

Implementa al menos dos mejoras y añade pruebas que demuestren su funcionamiento.

Nivel	Mejora	Concepto central
1	Editar el stock sin mutar el objeto original mediante with() o map().	inmutabilidad
1	Mostrar el producto de mayor valor de inventario.	reduce o toSorted
2	Agregar sede y representar existencias como matriz.	arreglo bidimensional
2	Persistir inventario en localStorage con JSON.	serialización
2	Mostrar conteo de productos por categoría en un Map.	Map y reduce
3	Importar fechas con reviver y validar su rango.	JSON.parse y Date
3	Agregar campos privados y método para ajustar stock.	class y #campo

14. Preguntas de reflexión

1. ¿Por qué const no vuelve inmutable el contenido de un objeto o arreglo?
2. ¿Qué diferencia existe entre alias = original y copia = { ...original }?
3. ¿Cuándo elegirías map() y cuándo forEach()?
4. ¿Por qué reduce() debe recibir un valor inicial en este laboratorio?
5. ¿Qué problema evita toSorted() frente a sort()?
6. ¿Por qué una función flecha no es apropiada para todos los métodos que usan this?
7. ¿Qué se pierde al serializar una instancia de clase y analizarla nuevamente?
8. ¿Cuándo Map es más expresivo que un objeto común?
9. ¿Por qué dos objetos { id: 1 } pueden coexistir en un Set?
10. ¿Qué validaciones deben realizarse después de JSON.parse()?

Cierre: La calidad no proviene de utilizar todas las estructuras a la vez, sino de asignar a cada una una responsabilidad coherente y mantener explícito cómo se transforma el estado.

Fuentes de consulta

- [ECMAScript 2026 Language Specification](#)
- [MDN - Working with objects](#)

- [MDN - Array](#)
- [MDN - Indexed collections](#)
- [MDN - Arrow function expressions](#)
- [MDN - Classes](#)
- [MDN - JSON](#)
- [MDN - Map](#)
- [MDN - Set](#)
- [MDN - structuredClone\(\)](#)